

Title: Lexical Analysis Using Lex/Flex

Objectives

- **Program 1:** Identify vowels and consonants in a given input text.
- **Program 2:** Recognize single-line (//) and multi-line (/*...*/) comments in a C program block.
- **Program 3:** Scan and distinguish capital (uppercase) and small (lowercase) letters.
- **Program 4:** Recognize core C programming language tokens: Keywords, Identifiers, Operators, and Numbers.
- **Program 5:** Match and classify standard and purely imaginary complex numbers (e.g., $5+3i$, $-4j$).
- **Program 6:** Count the total number of words and characters within an input string.

Theory

Lex is a computer program that generates lexical analyzers (scanners). It converts regular expressions into a deterministic finite automaton (DFA) written in C code (lex.yy.c).

Every Lex program follows a standard structure divided into three sections by `%%` markers:

Code snippet

Definitions (C header inclusions, variables, and regular expression aliases)
`%%`

Rules (Regular expressions matched with corresponding action blocks in C)
`%%`

User Subroutines (Main implementation and helper functions like `yylex()` and `yywrap()`)

- **Pattern Matching (Programs 1, 3, 5):** Standard character classes like `[A-Z]` and `[a-z]` filter alphabetic layout patterns, while patterns with optional signs `[+ -]?` and digit groupings `[0-9]+` capture complex mathematical constants.
- **Context Filtering (Program 2):** Lex uses string matching states or dedicated greedy look-ahead expressions to identify non-executable comment segments and exclude them or format them independently.
- **Lexical Tokens & Bookkeeping (Programs 4 & 6):** Lex automatically exposes internal variables such as `yytext` (the actual text matched by the pattern) and `yyleng` (the length of the matched text string) to make tracking character arrays and maintaining variable item counts seamless.

Programs

Program 1: Identify Vowels and Consonants

```
%{  
#include <stdio.h>  
%}  
  
%%  
[aeiouAEIOU] { printf("%s is a Vowel\n", yytext); }  
[a-zA-Z]      { printf("%s is a Consonant\n", yytext); }  
[^a-zA-Z\n]    { printf("%s is not a letter\n", yytext); }  
\n            { return 0; }  
%%  
  
int main() {  
    printf("Enter a string: ");  
    yylex();  
    return 0;  
}  
  
int yywrap() { return 1; }
```

Output:

```
(dharmdev@kali)-[~/6thSem/compiler_design/lab_1_google_classroom/vowel_and_consonant]  
$ ./a.out  
Enter a string: Dharmdev  
D is a Consonant  
h is a Consonant  
a is a Vowel  
r is a Consonant  
m is a Consonant  
d is a Consonant  
e is a Vowel  
v is a Consonant  
(dharmdev@kali)-[~/6thSem/compiler_design/lab_1_google_classroom/vowel_and_consonant]  
$
```

Program 2: Identify Single-Line and Multi-Line Comments

```
%{
#include <stdio.h>
%}

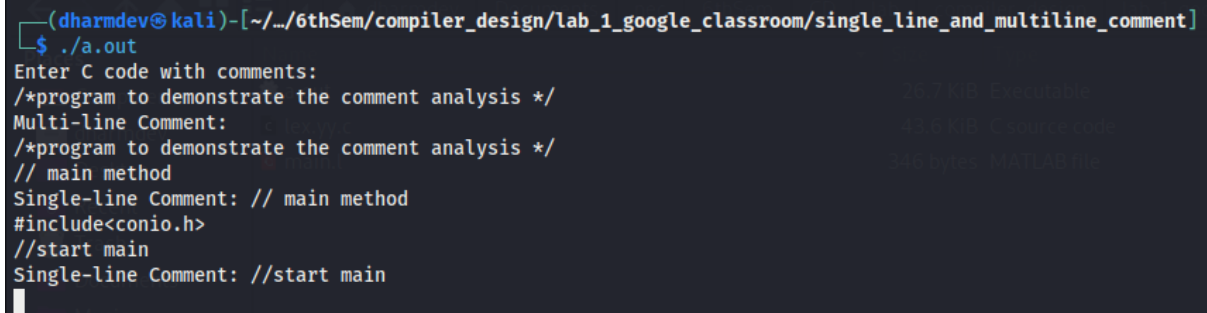
%%

"/".* { printf("Single-line Comment: %s\n", yytext); }
"^\*"([^\*]|\\*+([^\*]/))*\*+"/" { printf("Multi-line Comment:\n%s\n", yytext); }
.\n      { /* Ignore standard code code */ }
%%

int main() {
    printf("Enter C code with comments:\n");
    yylex();
    return 0;
}

int yywrap() { return 1; }
```

Output:

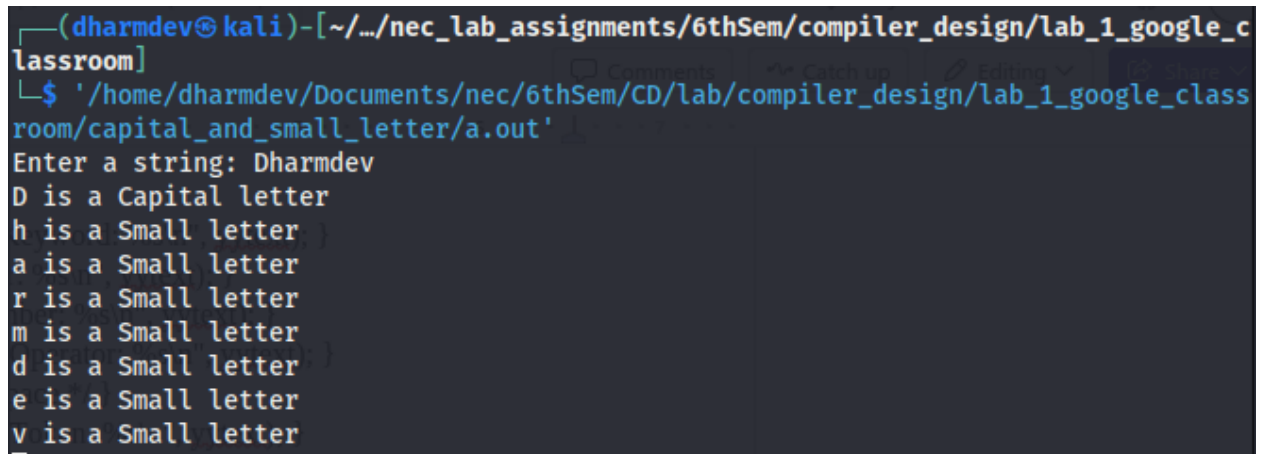


```
(dharmdev@kali)-[~/6thSem/compiler_design/lab_1_google_classroom/single_line_and_multiline_comment]
$ ./a.out
Enter C code with comments:
/*program to demonstrate the comment analysis */
Multi-line Comment:
/*program to demonstrate the comment analysis */
// main method
Single-line Comment: // main method
#include<conio.h>
//start main
Single-line Comment: //start main
```

Program 3: Identify Capital and Small Letters

```
%{  
#include <stdio.h>  
%}  
  
%%  
[A-Z] { printf("%s is a Capital letter\n", yytext); }  
[a-z] { printf("%s is a Small letter\n", yytext); }  
.|\\n { /* Ignore numbers, spaces, and punctuation */ }  
%%  
  
int main() {  
    printf("Enter a string: ");  
    yylex();  
    return 0;  
}  
  
int yywrap() { return 1; }
```

Output:



```
(dharmdev@kali)-[~/.../nec_lab_assignments/6thSem/compiler_design/lab_1_google_c  
lassroom]  
└─$ '/home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_1_google_class  
room/capital_and_small_letter/a.out'  
Enter a string: Dharmdev  
D is a Capital letter  
h is a Small letter  
a is a Small letter  
r is a Small letter  
m is a Small letter  
d is a Small letter  
e is a Small letter  
v is a Small letter
```

Program 4: Identify Keywords, Identifiers, Operators, and Numbers

```
%{
#include <stdio.h>
}%

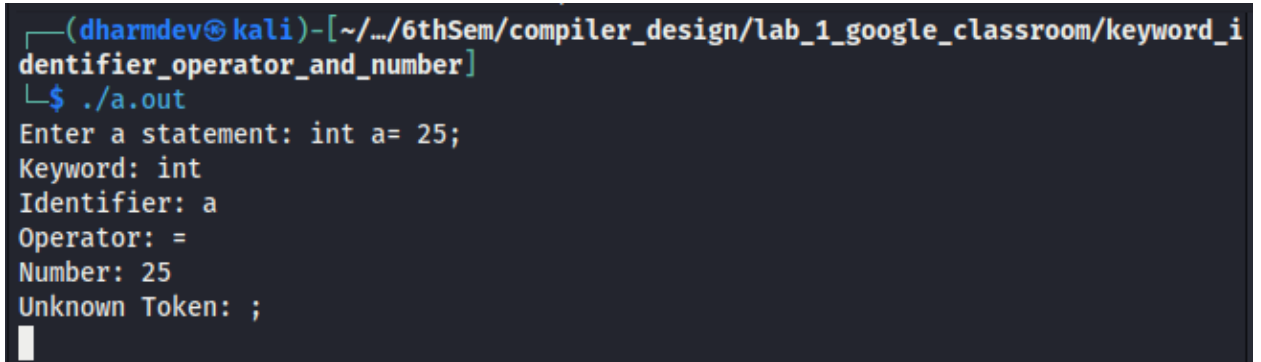
%%

if|else|while|int|float|return|char  { printf("Keyword: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("Identifier: %s\n", yytext); }
[0-9]+(\.[0-9]+)? { printf("Number: %s\n", yytext); }
"+"|"-"|"*"|" "/"|" "="|" "<"|" ">" { printf("Operator: %s\n", yytext); }
[ \t\n] { /* Ignore whitespace */ }
. { printf("Unknown Token: %s\n", yytext); }
%%

int main() {
    printf("Enter a statement: ");
    yylex();
    return 0;
}

int yywrap() { return 1; }
```

Output:



```
(dharmdev@kali)-[~/6thSem/compiler_design/lab_1_google_classroom/keyword_i
dentifier_operator_and_number]
└─$ ./a.out
Enter a statement: int a = 25;
Keyword: int
Identifier: a
Operator: =
Number: 25
Unknown Token: ;
█
```

Program 5: Identify Complex Numbers

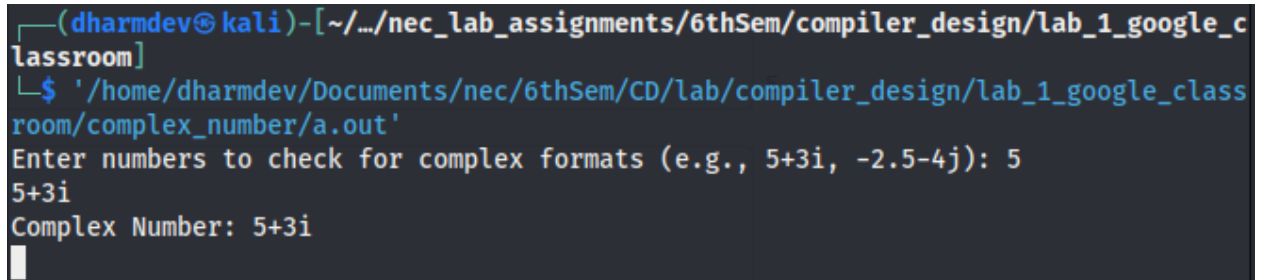
```
%{
#include <stdio.h>
%}

%%
[+-]?[0-9]+(\.[0-9]+)?[+-][0-9]+(\.[0-9]+)?[iI] { printf("Complex Number: %s\n",
yytext); }
[+-]?[0-9]+(\.[0-9]+)?[iI] { printf("Purely Imaginary Complex
Number: %s\n", yytext); }
.|\\n { /* Ignore other inputs */ }
%%

int main() {
    printf("Enter numbers to check formats (e.g., 5+3i, -2.5j): ");
    yylex();
    return 0;
}

int yywrap() { return 1; }
```

Output:



```
(dharmdev@kali)-[~/nec_lab_assignments/6thSem/compiler_design/lab_1_google_c
lassroom]
└─$ '/home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_1_google_class
room/complex_number/a.out'
Enter numbers to check for complex formats (e.g., 5+3i, -2.5-4j): 5
5+3i
Complex Number: 5+3i
█
```

Program 6: Count Characters and Words

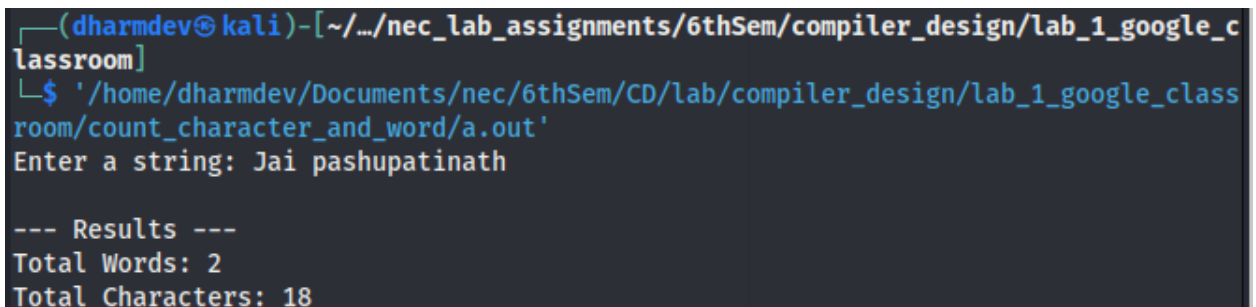
```
%{
#include <stdio.h>
int word_count = 0;
int char_count = 0;
%}

%%
[a-zA-Z0-9]+ { word_count++; char_count += yyleng; }
.           { char_count++; }
\n          { char_count++; return 0; }
%%

int main() {
    printf("Enter a string: ");
    yylex();
    printf("\n--- Results ---\n");
    printf("Total Words: %d\n", word_count);
    printf("Total Characters: %d\n", char_count);
    return 0;
}

int yywrap() { return 1; }
```

Output:



```
(dharmdev@kali)-[~/nec_lab_assignments/6thSem/compiler_design/lab_1_google_c
lassroom]
└─$ '/home/dharmdev/Documents/nec/6thSem/CD/lab/compiler_design/lab_1_google_class
room/count_character_and_word/a.out'
Enter a string: Jai pashupatinath

--- Results ---
Total Words: 2
Total Characters: 18
```

Conclusion

Using the Lex utility simplifies compiler scanner design by removing the need to manage input buffering, state machine transitions, or string tracking loops manually in C. By feeding declarative regular expressions directly into Lex, custom rule matching and lexical token isolation are automatically optimized into functional execution systems.